

This lecture explains the concept of encapsulation in Python, focusing on how it's used to hide information and control access to class attributes. Here's a breakdown:

- What is Encapsulation? Encapsulation is like wrapping something inside a capsule, hiding the internal details. In object-oriented programming (OOP), it involves restricting direct access to certain data or methods within a class.
-
- Public Variables: By default, variables in a Python class are public. This means they can be accessed and modified directly from outside the class.
-
- Protected Variables (Convention): Adding a single underscore (`_`) before a variable name (e.g., `_name`) indicates that it's a protected variable. Python treats this as a convention, suggesting that users should access it with care and respect its intended protection. It's a signal to other developers that the variable is intended for internal use within the class or its subclasses, but it doesn't strictly prevent external access.
-
- Private Variables: Using double underscores (`__`) before a variable name (e.g., `__name`) signifies a private variable. Python implements name mangling to make these variables harder (but not impossible) to access directly from outside the class.
-
- Name Mangling: Python's name mangling transforms the variable name to `_ClassName__variable` (e.g., `_Student__name`). This mechanism helps to avoid naming conflicts in subclasses and provides a stronger level of encapsulation compared to protected variables. While it makes direct access more difficult, it's still possible to access the variable using the mangled name.
-
- Accessing Private Variables: Although direct access to private variables is discouraged, there are two ways to access them:
 - Name Mangling: Access the variable using the `_ClassName__variable` format.
 - Getter and Setter Methods: Create methods (getter and setter) within the class to control access to the private variable. This is the preferred approach as it allows you to encapsulate the logic for accessing and modifying the variable.
-
- Getter Methods: A getter method (e.g., `get_name`) is a function within the class that returns the value of a private variable. It provides a controlled way to access the variable's value without directly exposing it.
-
- Python's Philosophy: Python relies on the maturity of developers to respect the conventions of protected and private variables. It assumes that developers will understand the intended use of these variables and avoid accessing them inappropriately.
-
- Key Takeaway: Encapsulation in Python is primarily achieved through naming conventions (single and double underscores). While Python doesn't enforce strict access control like some other languages, it provides mechanisms to indicate the

intended visibility of variables and encourages developers to follow these conventions for better code organization and maintainability.

-